

PRODUCT STRATEGY / BUILD PLAN / GO-TO-MARKET

Passoff

The complete business, product, technical and launch playbook for a branded client approval and asset handoff platform.

Recommended wedge

Final review, formal sign-off and professional handoff for freelance web designers and micro-agencies.

Prepared for product validation and implementation

Research and pricing checked August 29, 2026

Executive summary

Passoff should not enter the market as another general-purpose annotation tool. The strongest opening is a focused, branded room that helps a freelancer finish a project: collect final feedback, capture a defensible approval, freeze the approved version, and deliver final assets through one permanent link.

1 niche

Start narrow

Webflow freelancers and small web studios.

\$19/\$49

Core pricing

Solo and Studio plans, billed per workspace.

12 weeks

MVP target

A production-ready scope with controlled risk.

The strategic decision

Sell the outcome - a clean final yes and a professional handoff - not the mechanics of pins, uploads or client portals. Pin commenting is necessary, but it is not the moat.

Recommended offer

- A client opens one branded link without creating an account.
- The client reviews images, PDFs and website snapshots, then leaves contextual feedback.
- Approval records the exact revision, approver identity, timestamp and acceptance statement.
- Approved work is locked. Any later request becomes a new revision round.
- Final files, launch notes and access instructions remain available in an organized handoff room.

What success looks like

Within the first 90 days, validate five paying design businesses, run at least ten real client projects, achieve reliable client-side activation without training, and produce three case studies showing faster approval or fewer revision cycles. Do not expand into invoicing, CRM or broad project management until this loop is working.

Contents

Part	Section	Coverage
01	Business strategy	Market thesis, customers, product options, competition and pricing
02	Go-to-market	Messaging, acquisition, content, launch plan and operating metrics
03	Product specification	MVP, user journeys, lifecycle and acceptance criteria
04	Technical design	Architecture, data model, APIs, security, storage and operations
05	Delivery roadmap	Sprints, tests, risks, launch gates and developer handoff expansion
A	Sources	First-party competitor and infrastructure references



Business strategy

Positioning, customer, business model and competitive reality

1. Market thesis and positioning

The customer does not primarily buy collaboration software. They buy reduced ambiguity, faster payment, fewer unpaid revisions and a more professional client experience. The job begins near the end of a project, when work is almost finished but the freelancer still lacks a clean decision and a durable delivery record.

The core job to be done

Customer statement

When my project is ready for final review, help me collect precise feedback, obtain an explicit sign-off and deliver everything professionally, so I can close the project without chasing email threads or reopening settled work.

Positioning statement

For freelance web designers and small creative studios that struggle with final feedback and handoff, Passoff is a branded client approval room that turns one share link into contextual review, recorded sign-off and permanent delivery. Unlike broad proofing or project-management products, Passoff is designed around the final 10 percent of client work.

Recommended category language

Use	Avoid	Reason
Client approval and handoff	Project management	Names the end result and avoids a crowded replacement category.
Branded approval room	Generic client portal	Feels lighter and does not imply billing, CRM or messaging.
Final sign-off	E-signature platform	Signals a workflow record without overclaiming legal equivalence.
Visual feedback	Bug tracker	Keeps the experience client-friendly and nontechnical.

A note on legal claims

Passoff can preserve evidence of a business approval, but marketing should not promise that every approval is a legally binding electronic signature in every jurisdiction. Terms, consent language and evidence retention should be reviewed by qualified counsel before making that claim.

2. Customer segments and buying triggers

Begin with one narrow ideal customer profile, then expand horizontally after repeatable acquisition and retention are proven.

Segment	Pain	Buying trigger	Priority
Solo Webflow designer	Email approvals and late changes	A project is approaching launch	Primary
2-10 person web studio	Conflicting stakeholder feedback	Several simultaneous client reviews	Primary
Brand identity designer	Messy final asset delivery	A logo or brand package is ready	Secondary
WordPress agency	Website QA mixed with client approval	Staging review and launch	Secondary
Video production team	Frame-specific review	Creative review cycle	Defer - strong incumbents
Enterprise marketing team	Compliance-heavy proofing	Procurement-led evaluation	Defer - long sales cycle

Qualification signals

- Completes at least three client projects per month or charges roughly \$3,000 or more per project.
- Uses email, Slack, Google Drive or ad hoc forms for final approval.
- Has experienced client ghosting, conflicting comments, scope creep or disputes about what was approved.
- Cares about brand presentation and is comfortable paying for small workflow tools.
- Does not want to force clients into another account or workspace.

Interview prompts

- 1 Walk me through the last project from final draft to payment and handoff.
- 2 Where did approval actually happen, and how would you prove what was approved?
- 3 What usually causes one more revision round?
- 4 What do you send after launch, and how often do clients ask for it again?
- 5 What have you tried, what did you dislike, and what would this problem be worth if it disappeared?

Interview rule

Ask about a completed project and observable behavior. Do not ask whether someone 'would use' a hypothetical product until you understand what they currently do and pay for.

3. Product direction options

Direction	Benefit	Risk	Decision
Visual feedback tool	Easy to explain and demonstrate	Crowded; pins and comments are commoditized	Do not lead here
Full agency portal	High potential contract value	Large scope: billing, CRM, tasks, messaging and permissions	Future expansion only
Approval and handoff room	Clear wedge tied to revenue and scope control	Must make sign-off and handoff excellent	Recommended
Vertical Webflow add-on	Concentrated distribution and messaging	Platform dependence	Recommended launch niche

The initial wedge

Passoff should cover a complete narrow loop: create room, add review items, share, receive contextual feedback, publish a revision, collect approval, lock the signed revision, and deliver final assets. A thin but complete loop is more valuable than disconnected features across the entire agency lifecycle.

Expansion sequence

- 1 Web project final review and handoff.
- 2 Developer handoff generated from a client-approved revision.
- 3 Branding and design package templates.
- 4 Reusable agency templates and team workflows.
- 5 Integrations with Webflow, Framer, Figma, Drive, GitHub, Linear and Jira.
- 6 Optional deposits, final invoices or recurring care-plan conversion.

4. Competitive landscape

The category is validated but crowded. Pastel is the closest product-level threat; BugHerd and Ruttl are stronger in website QA; MarkUp.io offers broad simplicity; Frame.io owns professional video review; Filestage serves structured enterprise proofing; ManyRequests serves broader agency operations.

Product	Published entry price	Primary strength	Passoff opening
Ruttl	Free; Pro \$18/user/mo	Websites, apps, images and PDFs	Handoff and approval record
Pastel	Free; Pro \$35 mo or \$29 annual	Closest overlap; no-login feedback and approvals	Formal closeout and delivery
BugHerd	\$50 mo or \$42 annual	Website QA, metadata and task board	Simpler client-facing finish
MarkUp.io	\$79/mo	Unlimited users/projects; broad formats	Affordable solo positioning
Frame.io	Pro \$15/member/mo	Video review and secure sharing	Non-video creative handoff
Filestage	Starter \$199/mo	Structured enterprise proofing	Freelancer price and speed
ManyRequests	\$59 mo or \$39 annual	Full portal, billing and operations	No system replacement required
Atarim	\$67/mo	Website collaboration and AI agency workflows	Focused end-of-project workflow

Prices are published US prices observed August 29, 2026. Annual equivalents generally require annual payment. Always verify before using this table externally.

Competitive conclusion

Do not compete on pins

Passoff wins if the workflow produces a clear business outcome: the client approves a specific revision, new changes are visibly out of scope, and final deliverables stay organized. Confetti is delight; the approval ledger is value.

Defensible advantages to build

- End-to-end closeout templates for specific project types.
- A reliable approval evidence model with immutable revision history.
- A polished, white-labeled client experience at freelancer-friendly pricing.
- A viral distribution loop carried by every shared room and approval receipt.
- Workflow data that enables useful benchmarks such as average days to approval.

5. Packaging, pricing and economics

Charge for active client rooms and workspace capability, not for external clients. Unlimited reviewers should be standard because client participation creates value and distribution.

Plan	Price	Capacity	Key value
Trial	\$0 for 14 days	1-2 live projects	Full product; no card; trial badge
Solo	\$19 monthly / \$15 annual	5 active rooms; 10 GB	Basic branding; unlimited clients
Studio	\$49 monthly / \$39 annual	25 active rooms; 100 GB	Full branding; 3 team members; approval records
Agency	\$99 monthly / \$79 annual	100 active rooms; 500 GB	Custom domain; no vendor badge; 10 members
Enterprise	Custom	Negotiated	SSO, retention policy, audit export and SLA

Pricing principles

- Make clients and reviewers free. Charging them suppresses the core workflow.
- Do not promise unlimited storage. Use explicit quotas, file-size limits and retention policies.
- Archived rooms can remain readable while not counting against active-room limits.
- Keep vendor attribution on Trial and Solo; remove it on higher tiers.
- Offer the first 50 customers a \$12 monthly founding rate, but avoid lifetime deals.

Illustrative unit economics

Scenario	Assumption	Illustrative result
Solo payment	\$19 card payment at 2.9% + \$0.30	About \$18.15 before infrastructure and tax
Studio payment	\$49 card payment at 2.9% + \$0.30	About \$47.28 before infrastructure and tax
100 customers	70 Solo + 25 Studio + 5 Agency	\$3,053 MRR before discounts/churn
500 customers	Same plan mix	\$15,265 MRR before discounts/churn

Payment examples use Stripe's published standard domestic-card rate of 2.9% + \$0.30 as of the research date. Tax, disputes, international cards and subscription-specific products can change the actual result.



Go-to-market

Message, acquisition channels, launch plan and operating metrics

6. Marketing strategy

Own a painful moment rather than a software category. The lead message should connect approval to cash flow, scope control and professionalism.

Homepage message

Recommended hero

Get the final yes in writing. Collect visual feedback, lock client approval and deliver every website through one beautifully branded link.

Supporting proof

- Clients never create an account.
- Every comment points to the exact location and revision.
- Approval records the person, time, statement and content digest.
- New requests after approval open a new revision round.
- The handoff page remains available when the client needs assets later.

Message tests

Angle	Headline	What it tests
Approval	Stop chasing clients for final approval.	Urgency and cash-flow pain
Scope	Make 'one last change' a new revision - automatically.	Scope-creep pain
Brand	Make every project handoff look as good as the work.	Professional identity

Use interview and conversion data to select the primary angle. Keep the others as campaign themes rather than combining all three into one crowded hero.

7. Acquisition system

The first customers should come from high-touch outreach and communities. Search and partnerships become compounding channels after the product has evidence.

Channel	First action	Offer	When to scale
Founder outreach	10 personalized contacts/day	Set up the next live project	Immediately
Free toolkit	Publish sign-off and handoff templates	Download plus working demo	Immediately
Communities	Answer approval and scope questions	Useful workflow, no link drop first	Immediately
SEO	Publish six high-intent pages	Template plus product CTA	After message validation
Partners	Recruit educators and template sellers	25% for first 12 months	After 5 happy customers
Launch sites	Coordinate one public launch	Demo, proof and founder offer	After 3 case studies
Paid media	Retarget qualified visitors	Trial or template	Only after organic conversion

Founder outreach example

Short message

Hi Sarah - I saw the site you launched for Acme. How do you collect final approval and hand off the completed project today? I am building a branded approval room for small studios. I would be happy to set up your next real project at no cost in exchange for candid feedback.

Product-led distribution

- Place a discreet 'Delivered with Passoff' footer on Trial and Solo rooms.
- Add the same attribution to approval receipts without weakening the agency's brand.
- After approval, invite clients who also deliver services to create their own room.
- Give agencies a referral link and credit, then measure signups by originating room.

8. Content and search plan

Create content around the exact moment a freelancer realizes the current workflow is failing. Each page should include a practical asset, a live product example and one next action.

Intent	Content	Primary CTA
Template	Website client sign-off template	Use it in a free approval room
Checklist	Website handoff checklist	Generate a branded checklist
Problem	How to stop final-round scope creep	See the revision lock workflow
Workflow	How to hand off a Webflow website	Start the Webflow room template
Category	Client approval software for designers	Compare and start trial
Alternative	Pastel alternative for freelancers	See honest feature comparison
Alternative	MarkUp.io alternative for small agencies	Try an affordable handoff room
Education	What counts as client approval?	Download approval language

Weekly founder cadence

- One short before-and-after workflow video.
- Two educational LinkedIn or X posts.
- One article, template or teardown.
- Fifty personalized prospect conversations.
- One recorded customer interview and one product observation session.

Free Client Sign-Off Kit

Bundle an approval email, final-revision reminder, website handoff checklist, scope-creep response, approval language and a sample room. The kit should solve a real problem even if the reader never buys, while naturally demonstrating why a live, versioned approval is safer than an email template.

9. Ninety-day launch plan

Timing	Phase	Deliverables	Exit criterion
Days 1-15	Discovery	20 interviews; 5 observed workflows; landing page	3 people agree to pilot a live project
Days 16-30	Concierge pilot	Manually configure rooms; test messages and pricing	5 real projects; 2 paid commitments
Days 31-60	Private beta	Core workflow, notifications, approval ledger, billing	10-20 businesses; 3 case studies
Days 61-75	Acquisition base	Six SEO pages; toolkit; partner outreach	Repeatable qualified signups
Days 76-90	Public launch	Demo, customer proof, referral system, launch campaign	5+ paying accounts and healthy activation

Operating scorecard

Metric	Initial target	Diagnostic meaning
Signup to room created	>= 50%	Onboarding and initial value clarity
Room created to shared in 24h	>= 60%	True activation, not account creation
Shared room viewed	>= 70%	Delivery and client friction
Viewed room reaches feedback/approval	>= 50%	Client-side usability and urgency
Qualified trial to paid	>= 15%	Offer, pricing and product value
CAC payback	< 3 months	Sustainable acquisition economics
Signups from shared-room loop	25% by month 6	Built-in distribution strength

These are operating targets, not universal benchmarks. Segment results by source, project type and plan so averages do not hide a strong niche or a broken channel.



Product specification

MVP scope, client flows, state model and acceptance criteria

10. MVP scope and boundaries

The MVP must complete the approval-and-handoff loop with production-grade security. It does not need to run the agency.

Must ship	Can follow	Explicitly out of scope
Agency authentication	Developer handoff rooms	CRM
Project and revision creation	Custom domains	Invoicing and proposals
Private asset upload	Video timecode comments	Full task management
Magic-link client access	Figma/Drive integrations	Chat and meetings
Image/PDF/URL review	Agency templates	Website builder
Percentage-based pins	Exports and webhooks	Enterprise SSO
Comments and resolution	Password-protected links	Automated design critique
Revisioned approval	Approval certificates	Native mobile apps
Handoff files and notes	Analytics dashboard	Client billing portal
Transactional email	Referral program	Public marketplace

MVP release gate

- A new agency user can publish and share a room in under five minutes.
- A first-time client can view, comment and approve without instructions or an account.
- Every approval is tied to a frozen revision and cannot be silently changed.
- Private files are never publicly enumerable and are served through short-lived access URLs.
- Notifications are idempotent and do not send duplicates on retries.
- The application passes accessibility, mobile, security and recovery checks defined later in this playbook.

11. User journeys

Agency journey

- 1 Create an account and configure business name, logo, accent color and reply-to email.
- 2 Create a project from a template and enter client/project names.
- 3 Upload assets or add a review URL; group them into a revision.
- 4 Preview the exact client experience and publish the revision.
- 5 Send the magic link from Passoff or copy it into the agency's own message.
- 6 Receive view and comment notifications; resolve feedback and publish a new revision.
- 7 Request final approval with a clear acceptance statement.
- 8 After approval, add final files and handoff notes, then archive the room.

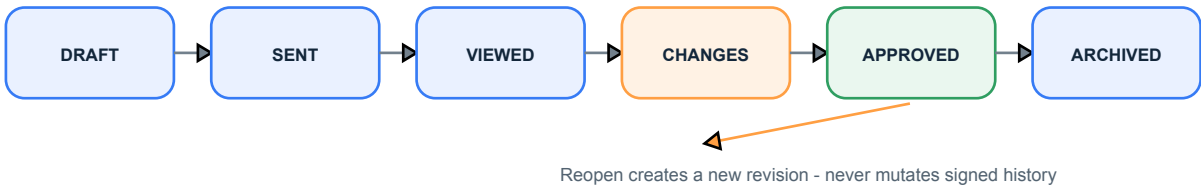
Client journey

- 1 Open a secure link and optionally enter name/email if the agency requires identification.
- 2 Review only the currently published revision and navigate its assets.
- 3 Click an image or page location, enter a concise comment and submit.
- 4 Review unresolved feedback and the agency's replies.
- 5 Choose 'Request changes' or 'Approve this revision'.
- 6 Confirm approval in a second step that names the project and revision.
- 7 Receive an approval receipt and access the handoff area when the agency releases it.

Anti-confusion rules

- Never show draft assets to a client.
- Never move an existing pin when a new revision has different dimensions; preserve its original revision context.
- Never treat a page view or resolved comment as approval.
- Never edit the content of a completed approval record; append a superseding event instead.
- Always show whether the client is viewing the latest published revision.

12. Project lifecycle and domain rules



A project may cycle between feedback and published revisions. Approval freezes one revision; reopening creates a new revision and preserves the prior approval record.

Key domain invariants

Invariant	Enforcement
Only a published revision can be reviewed	Database constraint plus server authorization
Only the current published revision can accept new feedback	Revision status check on every mutation
Approval points to one immutable revision	Foreign key plus frozen content digest
Approved content cannot be replaced	Create a new revision; never update asset membership
Magic links are revocable and expirable	Store token hash, status and expiry
Client identity is explicit at approval	Require name/email or verified one-time code by workspace policy
Every sensitive mutation is auditable	Append-only event row written in the same transaction

Recommended statuses

Project: DRAFT, SENT, VIEWED, CHANGES_REQUESTED, APPROVED, ARCHIVED. Revision: DRAFT, PUBLISHED, SUPERSEDED, APPROVED. Comment: OPEN, RESOLVED, WONT_FIX. Share link: ACTIVE, REVOKED, EXPIRED.

13. Detailed acceptance criteria

Capability	Acceptance criteria
Create project	Valid owner can create; default status DRAFT; unique ID returned; audit event written.
Upload asset	Direct upload uses allowed MIME/size; checksum stored; failed upload leaves no usable asset.
Publish revision	At least one ready asset; membership becomes immutable; previous revision is superseded.
Open magic link	Raw token is never stored; expired/revoked tokens fail generically; first valid view is recorded once.
Drop pin	Coordinates are clamped to 0-1; asset/revision is current; comment text is sanitized and rate-limited.
Resolve comment	Agency member only; resolution event records actor and timestamp; client sees updated state.
Approve	Two-step confirmation; identity and statement captured; digest matches revision; transaction is atomic.
Reopen	Prior approval remains; new revision begins in DRAFT; client sees that approval was superseded.
Download handoff	Permission checked per request; signed URL expires; download event may be recorded.
Delete workspace	Retention policy applied; billing canceled; export/notice path documented; assets scheduled for purge.



Technical design

Architecture, schema, APIs, security, storage and operational controls

14. Recommended implementation stack

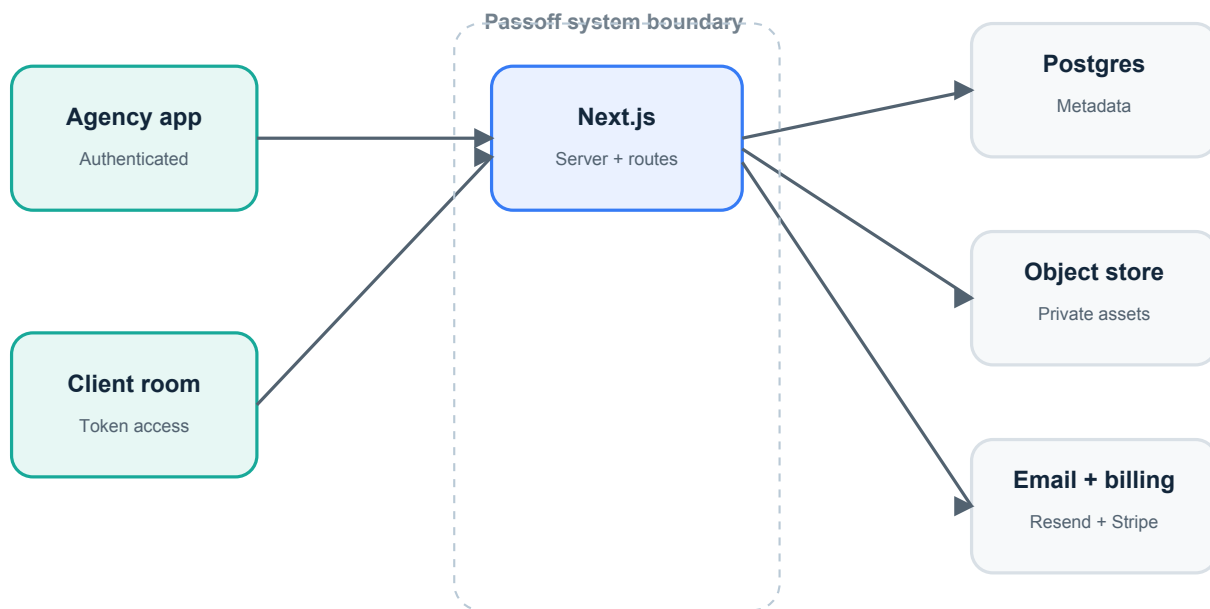
The provided Passoff starter currently uses Next.js 16.3.3, React 19.2.8, TypeScript, Tailwind CSS 4, shadcn/Base UI, Drizzle ORM and libSQL. That is an excellent UI and local-development base. For a multi-tenant production SaaS, use managed Postgres for the primary database and private object storage for assets.

Layer	Recommendation	Purpose
Application	Next.js App Router + TypeScript	Server-rendered dashboard, public client rooms and route handlers
UI	React, Tailwind CSS, shadcn/Base UI	Accessible design system and responsive client canvas
Database	Postgres + Drizzle ORM	Relational integrity, transactions, migrations and audit history
Authentication	Auth.js or managed provider	Agency sessions; clients use scoped share tokens
Object storage	Vercel Blob, Cloudflare R2 or S3	Private files, signed URLs and multipart uploads
Email	Resend	Share, view, comment, approval and receipt notifications
Billing	Stripe Checkout + Customer Portal	Trials, subscriptions, taxes and lifecycle webhooks
Hosting	Vercel	Next.js deployment, previews, domains and observability
Monitoring	Structured logs + error tracking	Request IDs, alerts, webhook reconciliation and audit

Repository implementation rule

Before writing application code, read the Next.js 16.3.3 documentation shipped in node_modules, because framework APIs and conventions may differ from older examples. Keep Drizzle schema and SQL migrations together, and require a passing production build before release.

15. System architecture



Recommended logical architecture. Clients never receive database or object-store credentials; all sensitive access is mediated by server-side authorization and short-lived signed URLs.

Request boundaries

- Server components load authenticated dashboard data directly through the data-access layer.
- Route handlers accept uploads, public feedback, approval commands and provider webhooks.
- Every public-room request resolves the share token hash to workspace, project and permission scope.
- The browser uploads large files directly to object storage using a short-lived signed upload grant.
- A job or transactional outbox handles email and external side effects after the database commit.

Tenant isolation

Every tenant-owned row carries `workspace_id`. Server queries require an explicit workspace scope; never fetch by a user-controlled object ID alone. Where practical, add Postgres row-level security as a second line of defense, but do not treat it as a substitute for application authorization.

16. Relational data model

Table	Important fields	Role
users	id, email, name, created_at	Agency identity
workspaces	id, name, slug, logo_key, brand_color, plan	Tenant and branding
memberships	workspace_id, user_id, role	OWNER, ADMIN, MEMBER
projects	id, workspace_id, client_name, name, status	Top-level client room
revisions	id, project_id, number, status, published_at, digest	Immutable review versions
assets	id, workspace_id, kind, object_key, mime, bytes, checksum	Stored file or external link
revision_assets	revision_id, asset_id, sort_order, display_meta	Frozen revision membership
share_links	id, project_id, token_hash, scope, expires_at, revoked_at	Public authorization
reviewers	id, project_id, email, name, verified_at	Client identity
comments	id, revision_asset_id, reviewer_id, x, y, body, status	Contextual feedback
approvals	id, revision_id, reviewer_id, statement, digest, approved_at	Immutable sign-off
handoff_items	id, project_id, asset_id/link, label, category	Final delivery
audit_events	id, workspace_id, actor, action, target, metadata, at	Append-only history
outbox_events	id, type, payload, available_at, processed_at, attempts	Reliable side effects
subscriptions	workspace_id, provider IDs, plan, status, period_end	Billing mirror

Minimum indexes

- projects(workspace_id, status, updated_at desc)
- revisions(project_id, number desc) unique on project_id + number
- share_links(token_hash) unique and share_links(project_id, revoked_at)
- comments(revision_asset_id, created_at) and comments(status)
- audit_events(workspace_id, created_at desc)
- outbox_events(processed_at, available_at) partial where processed_at is null

17. Drizzle schema example

The production schema should be split by domain and accompanied by idempotent SQL migrations. The fragment below illustrates the approval relationship, not a complete migration.

```
export const revisions = pgTable("revisions", {
  id: uuid("id").defaultRandom().primaryKey(),
  projectId: uuid("project_id").notNull()
    .references(() => projects.id, { onDelete: "cascade" }),
  number: integer("number").notNull(),
  status: revisionStatus("status").notNull().default("DRAFT"),
  contentDigest: text("content_digest"),
  publishedAt: timestamp("published_at", { withTimezone: true }),
}, (t) => [unique().on(t.projectId, t.number)]);

export const approvals = pgTable("approvals", {
  id: uuid("id").defaultRandom().primaryKey(),
  revisionId: uuid("revision_id").notNull()
    .references(() => revisions.id, { onDelete: "restrict" }),
  reviewerId: uuid("reviewer_id").notNull()
    .references(() => reviewers.id, { onDelete: "restrict" }),
  acceptanceStatement: text("acceptance_statement").notNull(),
  contentDigest: text("content_digest").notNull(),
  approvedAt: timestamp("approved_at", { withTimezone: true })
    .defaultNow().notNull(),
});
```

Atomic approval transaction

Lock the revision row, verify it is the currently published revision, recompute or verify the digest, insert the approval, update project/revision status, append the audit event and enqueue the receipt in one database transaction.

18. API and server-action surface

Endpoint	Authorization	Purpose
POST /api/projects	Agency session	Create project
POST /api/projects/:id/revisions	Agency + workspace	Create draft revision
POST /api/uploads/grant	Agency + quota	Issue signed direct-upload grant
POST /api/revisions/:id/publish	Agency + owner role	Freeze and publish revision
POST /api/projects/:id/share-links	Agency + project	Create or rotate share token
GET /share/:token	Share token	Render public room
POST /api/public/:token/view	Share token + rate limit	Record first/meaningful view
POST /api/public/:token/comments	Share token + revision	Create contextual feedback
POST /api/public/:token/decision	Share token + identity	Request changes or approve
POST /api/stripe/webhook	Verified Stripe signature	Mirror subscription state
POST /api/resend/webhook	Verified provider signature	Track delivery/bounce state
POST /api/internal/outbox	Cron secret	Process queued side effects

Mutation pattern

- Parse and validate input with a shared schema.
- Authenticate the agency session or resolve the share token.
- Authorize against workspace, project, revision and capability.
- Enforce quotas, state transitions, idempotency key and rate limits.
- Write domain state, audit event and outbox event atomically.
- Return a stable typed response without leaking internal errors.

19. Magic-link security

A magic link is a bearer credential. Whoever possesses it may exercise its permissions, so the implementation must treat it like a password even though the client experience feels frictionless.

- Generate at least 32 random bytes with a cryptographically secure generator and encode as base64url.
- Store only SHA-256(token) in the database. Show the raw token once in the URL.
- Resolve tokens using constant-time comparison where applicable and return generic failures.
- Include scope such as VIEW, COMMENT or APPROVE; avoid one global capability flag.
- Support expiration, manual revocation and rotation. Rotation invalidates the former link.
- Do not place tokens in analytics events, logs, error reports or third-party referrer headers.
- Set Referrer-Policy: no-referrer and prevent search indexing on all public rooms.
- Rate-limit view, identity, comment and decision endpoints independently.
- For higher-risk projects, allow password or one-time email verification before approval.

Token generation sketch

```
const token = randomBytes(32).toString("base64url");
const tokenHash = createHash("sha256").update(token).digest("hex");
// Store tokenHash, projectId, scope, expiry. Return token only once.
```

Important distinction

An unguessable URL prevents discovery, not forwarding. Agencies must be able to revoke access, and high-sensitivity workflows need an additional identity check.

20. File upload and delivery design

Use private object storage and direct browser uploads. The application server should authorize and sign the upload, but should not proxy large asset bodies unless required.

- 1 Client requests an upload grant with filename, size and MIME type.
- 2 Server checks agency session, workspace quota, per-file limit and allowed content type.
- 3 Server creates a pending asset row and returns a short-lived signed upload URL.
- 4 Browser uploads directly, preferably multipart for large files.
- 5 Client confirms completion; server verifies object metadata and records checksum/size.
- 6 Background processing generates thumbnails or PDF previews and marks the asset READY.
- 7 Downloads use short-lived signed URLs after project/share authorization.

Controls

Risk	Control
Storage abuse	Per-file, workspace and plan quotas; reject before signing upload
Malicious file	Allowlist MIME/extensions, inspect magic bytes, optional malware scan
Executable content	Force attachment for risky types; never serve user HTML on app origin
Broken uploads	Pending state, expiry and cleanup job for abandoned objects
Asset guessing	Opaque keys; private store; signed access only
High transfer cost	Thumbnails, caching, lifecycle rules and download limits
Deletion mismatch	Transactional tombstone plus idempotent purge job

Cloudflare R2 publishes standard storage at \$0.015/GB-month, free Internet egress, a 10 GB-month free allowance and operation-based charges. Vercel Blob is a simpler repository fit but has regionalized transfer pricing. Treat provider choice as an ADR based on expected download volume and operational simplicity.

21. Visual feedback implementation

Pins must remain accurate when an image is resized. Store coordinates as normalized values relative to the intrinsic asset display box, not screen pixels.

Coordinate formula

$x = \text{clamp}((\text{pointerX} - \text{assetLeft}) / \text{renderedAssetWidth}, 0, 1); y = \text{clamp}((\text{pointerY} - \text{assetTop}) / \text{renderedAssetHeight}, 0, 1)$. Render at $\text{left} + x * \text{currentWidth}$ and $\text{top} + y * \text{currentHeight}$.

Implementation rules

- Use the image's content box, excluding letterbox padding, as the coordinate space.
- Store `revision_asset_id` with each pin so it never floats onto a different version.
- For PDFs, also store page number and normalized page coordinates.
- For websites, MVP should use captured screenshots or an explicit review overlay integration. Arbitrary `iframe` embedding fails when sites set CSP or X-Frame-Options.
- Debounce view-state events but never debounce comment submission without user feedback.
- Make pins keyboard reachable and pair each pin with an ordered comment list for accessibility.
- Use optimistic UI only when retries are idempotent and failed comments remain visibly recoverable.

Website-review decision

Method	Pros	Cons	MVP choice
Iframe/proxy	Live navigation	CSP, auth, cookies, privacy and rendering complexity	No
Screenshot capture	Stable coordinates and versioning	Not interactive; capture infrastructure	Yes
Injected script	True live-site context	Customer installation and security review	Later
Browser extension	Works across protected sites	Client setup friction	Later

22. Approval integrity and audit evidence

Approval is the most important domain event. Design it as an append-only fact linked to immutable content, not a mutable status field.

Content digest

Create a canonical manifest containing project ID, revision ID/number, ordered asset IDs, object checksums, link destinations, relevant display metadata and acceptance statement version. Serialize deterministically and hash with SHA-256. Store the digest on publication and copy it into the approval row.

Evidence captured

- Workspace, project and revision identifiers.
- Approver name, email, verification state and reviewer ID.
- Exact acceptance statement and its version.
- Content digest and ordered asset manifest.
- Server timestamp, request ID and coarse network/user-agent evidence subject to privacy policy.
- Link ID and authentication method used.
- Any later superseding or reopening event, without deleting the original approval.

Approval command sequence

- 1 Show a dedicated confirmation screen naming the project and revision.
- 2 Require an unchecked confirmation box and explicit Approve action.
- 3 Verify token scope, identity policy, current revision and digest server-side.
- 4 Insert approval, transition revision/project, audit and outbox rows atomically.
- 5 Return a receipt ID and send matching receipts to client and agency.
- 6 If a later change is necessary, create a new revision and mark the project as reopened.

23. Notifications, billing and webhooks

External providers retry and can deliver events out of order. Webhook handlers must verify signatures, store provider event IDs and be idempotent.

Event	Recipient	Delivery rule
Room shared	Client	Immediate; one message per share action
First meaningful view	Agency	Once per reviewer/link window
New feedback	Agency	Digest closely spaced comments
Agency reply	Client	Immediate or digest by preference
Approval requested	Client	Immediate with deadline/reminder schedule
Approved	Both	Immediate immutable receipt
Link revoked/expired	Agency	Dashboard event; email only if requested
Payment failed	Billing owner	Provider-driven lifecycle sequence

Billing architecture

- Stripe is the source for money movement; the database mirrors entitlement state for fast authorization.
- Use Checkout for signup and the Customer Portal for card, invoice and cancellation management.
- Map Stripe price IDs to internal stable plan keys. Do not scatter price IDs through UI code.
- Process subscription.created/updated/deleted and invoice payment events idempotently.
- Define downgrade behavior before launch: new uploads blocked, existing rooms readable, grace period explicit.
- Reconcile subscription state on a schedule so missed webhooks do not create permanent drift.

24. Security, privacy and abuse checklist

Area	Minimum control
Authorization	Deny by default; workspace scope on every query; object-level tests
Sessions	Secure, HttpOnly, SameSite cookies; rotation; CSRF protection where relevant
Share links	Hashed tokens; scope; expiry; revocation; no logs or referrers
Input	Schema validation, length limits, sanitization and safe rendering
Uploads	MIME/magic-byte checks, quotas, private storage and malware option
Rate limits	Per IP, token, workspace and action; stricter approval/identity limits
Secrets	Environment/secret manager only; rotation and least privilege
Webhooks	Signature verification, raw body handling, replay and idempotency protection
Logging	Structured request IDs; redact tokens, emails where unnecessary and asset URLs
Backups	Automated DB backups; restore test; object retention strategy
Privacy	Data inventory, retention schedule, deletion/export path and processor DPAs
Abuse	Report path, takedown process, content limits and account suspension tooling

Legal documents before paid launch

- Terms of Service with acceptable-use, billing, storage and termination provisions.
- Privacy Policy naming subprocessors and data categories.
- Data Processing Addendum path for business customers.
- Copyright/takedown and abuse contact process.
- Retention and deletion schedule, including backups and archived rooms.
- Clear statement about what approval means and does not mean.

This playbook is product and technical guidance, not legal advice.

25. Infrastructure costs and capacity

A lean production deployment can begin inexpensively, but asset storage and transfer must be instrumented from day one. Published prices below are directional snapshots from August 29, 2026.

Service	Published/assumed cost	Role
Vercel Pro	\$20/month plus usage; \$20 usage credit	App hosting and deployment
Neon Launch	Usage based; typical example \$15/month	Production Postgres
Supabase Pro alternative	\$25/month	Postgres, auth and 100 GB storage bundle
Resend	Free 3,000 email/mo; Pro \$20 for 50,000	Transactional email
Cloudflare R2	\$0.015/GB-month standard; egress free	Asset storage alternative
Stripe	2.9% + \$0.30 domestic card	Payment processing
Error monitoring	\$0-\$30 early stage	Exceptions and alerting
Domain/DNS	\$1-\$3 monthly equivalent	Brand and email domain

Illustrative monthly platform budget

Stage	Customers	Budget range	Dominant variable
Private beta	0-20	\$20-\$80	Production hosting and email domain
Early revenue	20-100	\$60-\$250	Storage, transfer, previews and monitoring
Growing	100-500	\$200-\$1,200	Asset volume, downloads, database and support tooling

Ranges are planning allowances, not vendor quotes. Actual cost depends much more on file size, preview generation and download frequency than on database row count. Add per-workspace metering for stored bytes, upload bytes, downloads, emails and active rooms.

26. Deployment and operations

Environments

Environment	Data	Rule
Local	Local/test database and object-store emulator	No production secrets or client assets
Preview	Ephemeral database branch and isolated storage prefix	Automatic expiry; no real customer email
Staging	Persistent synthetic data	Release candidate, webhook and restore testing
Production	Real customer data	Protected branch, reviewed migrations and alerting

Release pipeline

- 1 Run formatting, lint, type checks and unit tests.
- 2 Build the Next.js application in production mode.
- 3 Apply schema changes to an isolated preview database and run integration tests.
- 4 Deploy preview; execute smoke tests for agency login and public-room flows.
- 5 Review migration backward compatibility and rollback/forward-fix plan.
- 6 Deploy production, run migrations with an explicit job and execute post-deploy smoke tests.
- 7 Monitor errors, latency, webhook backlog, failed email and approval success rate.

Operational alerts

- Public-room 5xx rate or latency crosses threshold.
- Failed uploads or preview jobs accumulate.
- Outbox age exceeds five minutes or retry count grows.
- Stripe/Resend webhook signature failures spike.
- Approval transaction failure rate exceeds normal baseline.
- Storage usage jumps abnormally by workspace or globally.
- Database backup or restore verification fails.



Delivery roadmap

Sprints, testing, launch gates, risks and next decisions

27. Twelve-week engineering plan

Timing	Theme	Exit deliverable
Week 1	Foundation	Environments, CI, design tokens, auth skeleton, schema conventions
Week 2	Tenancy	Workspaces, memberships, branding and authorization tests
Week 3	Projects	Project/revision CRUD, state machine, dashboard
Week 4	Assets	Direct private upload, quotas, image preview and cleanup
Week 5	Sharing	Hashed magic links, public layout, view tracking and noindex
Week 6	Feedback	Pins, PDF pages, comments, replies, resolution and accessibility
Week 7	Revisions	Publish/supersede workflow, digest, comparison and client clarity
Week 8	Approval	Identity, two-step confirmation, atomic ledger and receipts
Week 9	Handoff	Delivery sections, downloads, archive/read-only behavior
Week 10	Billing	Stripe trial/plans, webhooks, entitlements and downgrade behavior
Week 11	Hardening	Rate limits, abuse controls, restore test, load and mobile QA
Week 12	Beta release	Onboarding, analytics, docs, monitoring and pilot migration

Team assumption

This schedule assumes one experienced full-stack engineer with product/design support and disciplined scope. A part-time solo build is more realistically 16-24 weeks. Website proxying, video processing, custom domains and enterprise SSO are excluded.

28. Test strategy

Layer	Required coverage
Unit	State transitions, digest canonicalization, token hashing, quotas and coordinate math
Database	Tenant scoping, constraints, transactional approval and migration idempotency
Integration	Object-storage grants, email outbox, Stripe/Resend webhook retries
End-to-end	Create -> upload -> publish -> share -> comment -> revise -> approve -> handoff
Authorization	Cross-tenant IDs, revoked/expired links, stale revisions and privilege escalation
Accessibility	Keyboard pins, focus order, labels, contrast, zoom and screen-reader review
Responsive	Mobile review, tablet dashboard and rotated/zoomed assets
Performance	Large image/PDF, 100+ pins, concurrent room views and signed URL latency
Recovery	Database restore, object purge retry, outbox replay and provider outage

Critical end-to-end assertions

- A client cannot approve a superseded revision by replaying an old request.
- Changing an asset after publication changes the digest and requires a new revision.
- A duplicate approval request produces one approval and one receipt.
- A forwarded view-only token cannot comment or approve.
- A workspace can never read another workspace's project, asset metadata or audit event.
- Deleting an uploaded object does not leave a functioning stale download URL after expiry.

29. Risk register

Risk	Impact	Mitigation
Crowded category	High	Position around closeout/handoff; validate willingness to switch before build
Clients ignore link	High	Excellent email copy, no account, mobile UX and reminders
Approval legal overclaim	High	Precise language, immutable evidence and legal review
Asset cost/abuse	High	Quotas, direct upload, private storage, lifecycle and monitoring
Website embedding failure	High	Use screenshot-based MVP; defer proxy/extension
Scope expansion	High	Explicit non-goals and metric-based expansion gates
Low retention	Medium	Annual plan, reusable templates, archive value and multi-project focus
Security incident	High	Defense in depth, token hygiene, least privilege and response plan
Email deliverability	Medium	Verified domain, webhooks, suppression handling and in-app fallback
Provider lock-in	Medium	Adapter interfaces and storage/database export paths

Kill criteria

Pause or reposition if, after 30 qualified interviews and 10 concierge projects, fewer than five businesses agree to pay at least \$19/month, clients consistently prefer email despite a no-account flow, or the primary value is discovered to be generic annotation rather than approval/handoff. Evidence should decide whether to continue, not sunk engineering effort.

30. Launch readiness checklist

Product

- ☐ Happy path complete
- ☐ Empty/error/loading states
- ☐ Mobile client review
- ☐ Archive and reopen behavior

Security

- ☐ Authorization matrix tested
- ☐ Tokens redacted
- ☐ Rate limits enabled
- ☐ Webhook signatures verified

Data

- ☐ Migrations rehearsed
- ☐ Backups enabled
- ☐ Restore tested
- ☐ Deletion and export documented

Operations

- ☐ Alerts configured
- ☐ Outbox monitored
- ☐ Support inbox live
- ☐ Incident owner defined

Commercial

- ☐ Plans in Stripe
- ☐ Trial/downgrade tested
- ☐ Taxes reviewed
- ☐ Cancellation path clear

Legal

- ☐ Terms and Privacy live
- ☐ Approval language reviewed
- ☐ Subprocessors listed
- ☐ Abuse process ready

Marketing

- ☐ Demo room
- ☐ Three case studies
- ☐ Six search pages
- ☐ Lifecycle emails and analytics

31. Expansion: developer handoff

Developer handoff is the recommended first major product expansion after client approval is working. It extends the same authoritative revision into implementation without turning Passoff into a design-inspection tool, source-control system or issue tracker.

Expansion promise

Give developers the approved context, assets and decisions they need - not another design-inspection tool. The developer should always know what the client approved, which revision is authoritative, what must be built and what changed after handoff.

End-to-end workflow

Design review -> Client approval -> Developer handoff -> Implementation QA -> Final client sign-off.

- 1 A client approves a frozen revision in the Client Review room.
- 2 The agency generates a Developer Handoff room directly from that revision.
- 3 Passoff carries forward the approved asset manifest, revision digest and decision history.
- 4 The agency adds implementation notes, acceptance criteria, source links and final assets.
- 5 Developers acknowledge receipt, ask contextual questions and link implementation issues.
- 6 The agency reviews the built result against the approved revision during Implementation QA.
- 7 Material changes create a new revision and visibly supersede the previous handoff.

Recommended room types

Room type	Primary audience	Outcome
Client Review	Client stakeholders	Feedback, revision decision and formal approval
Developer Handoff	Developers and technical partners	Approved context, assets, requirements and acknowledgement
Launch Handoff	Client owner and operations team	Final files, access instructions, training and maintenance details

Developer handoff expansion specification

Handoff packet

Section	Contents
Approved source	Exact approved revision, digest, decision history and superseded-state warning
Design	Figma links, page/component references, responsive behavior and interaction notes
Assets	Images, icons, fonts, documents, copy and downloadable export bundles
Design system	Colors, typography, spacing, breakpoints and token references
Implementation	Page-level notes, acceptance criteria, definition of done and known limitations
Delivery links	Repository, staging, CMS, analytics and issue-tracker references
Collaboration	Developer-only questions, acknowledgement, resolution state and change alerts
QA	Built-versus-approved checks, implementation defects and final verification

Product boundaries

- Do not replicate Figma’s design-property inspection; deep-link to the relevant frame or component.
- Do not build a general issue tracker; create or link GitHub, Linear and Jira issues.
- Do not store passwords, API keys or recovery codes; link to a dedicated secrets manager such as 1Password.
- Do not allow developer notes to mutate the client-approved revision or its evidence record.
- Do preserve an explicit chain from approved client revision to handoff packet to implementation QA.

Data-model additions

Entity	Purpose
handoff_rooms	Type, source revision, status, released_at and superseded_at
handoff_sections	Ordered structured sections with visibility and completion state
implementation_items	Requirement, acceptance criteria, owner, status and source reference
external_issue_links	Provider, repository/team, external ID, URL and sync metadata
handoff_acknowledgements	Recipient identity, revision/digest and acknowledged_at
qa_findings	Approved source location, implementation location, severity and resolution

Developer handoff packaging and rollout

Packaging recommendation

Plan	Developer handoff capability
Solo	One simple handoff generated from an approved revision; vendor attribution remains
Studio	Unlimited handoff rooms within project limits, private team notes and three internal members
Agency	Custom templates, GitHub/Linear/Jira integrations, webhooks, API and audit export
Enterprise	SSO, retention policies, advanced access controls and integration governance

Expansion gate

Build only after the core loop is proven

Begin developer handoff when at least five to ten agencies repeatedly approve real projects in Passoff, at least three explicitly request a structured design-to-development handoff, and the approved revision model is stable. Validate the first handoffs manually before building integrations.

Success measures

- Percentage of approved projects that generate a developer handoff.
- Median time from client approval to developer acknowledgement.
- Percentage of implementation questions resolved inside the handoff context.
- Implementation defects traced to missing or ambiguous handoff information.
- Studio/Agency upgrades attributable to developer handoff capability.
- Projects returning for final QA and client sign-off after implementation.

Rollout sequence

- 1 Concierge handoff template and developer acknowledgement.
- 2 Structured sections, acceptance criteria and approved-revision inheritance.
- 3 Developer-only comments and implementation QA findings.
- 4 One-way issue creation for GitHub, Linear and Jira.
- 5 Webhooks and API export after usage patterns are stable.
- 6 Selective two-way synchronization only where customers demonstrate durable need.

32. Competitor architecture: how Ruttl reviews live sites

Ruttl appears to use a hybrid delivery model: a low-friction URL review experience for public websites, plus an installed script or browser extension for authenticated and technically complex applications. Ruttl does not publish its complete internal architecture, so the proxy details below are an evidence-based technical inference rather than a confirmed implementation disclosure.

Why motion continues to work

Ruttl is not recreating a site's animation. When the original CSS and JavaScript execute in the review environment - or the feedback layer runs directly on the real page - carousels, CSS animation, GIFs, video, canvas and most application behavior continue running.

Observed product paths

Path	Documented behavior	Likely technical model
Public website URL	Paste a URL; the page loads inside Ruttl with Comment, Browse and Edit modes.	Ruttl-controlled fetch/proxy or equivalent document-processing layer that preserves resources and injects review controls.
Web app script	Add Ruttl's plugin code before the closing body tag; a feedback widget loads on the application.	JavaScript executes directly on the actual site, preserving the viewer's session and the application's normal runtime.
Chrome extension	Open a logged-in or password-gated page and add it to a Ruttl web-app project.	The extension injects the review interface into the active browser tab and uses the user's existing authenticated session.

Evidence and inference boundary

- Confirmed: Ruttl says public pages load inside its project interface and support element-level comments, Browse mode and temporary visual edits.
- Confirmed: Ruttl documents script installation and a Chrome extension for live web apps, logged-in screens and password-gated pages.
- Confirmed: Ruttl lists login prompts, CAPTCHA, anti-bot controls, firewalls, broken CSS/scripts and interactive elements among reasons a page may fail to load.
- Inferred: the public-URL path behaves like a fetching/proxying and resource-rewriting system. Ruttl does not publicly specify its request pipeline, rewrite rules or isolation model.

Ruttl-style implementation model for Passoff

A public URL reviewer can retrieve the target document, normalize relative resource addresses, neutralize headers that prevent display where legally and contractually appropriate, inject the Passoff overlay, and deliver the result from a controlled origin. Original frontend code then produces the site's movement and interaction. This path is convenient but has a large compatibility and security burden.

Illustrative request flow

- 1
- A workspace member submits a public review URL and Passoff validates the protocol, host and network destination.
- 2
- A restricted fetch service retrieves the document while enforcing SSRF protections, timeouts, size limits and redirect limits.
- 3
- The service rewrites relative document, stylesheet, script, image and navigation references where required.
- 4
- Passoff injects a sandboxed overlay that switches between Browse and Comment modes.
- 5
- The browser runs permitted original CSS and JavaScript; Passoff records comments against URL, element identity, viewport and fallback coordinates.
- 6
- When the URL path cannot render safely or faithfully, Passoff offers the installed script, browser extension or screenshot fallback.

Illustrative comment anchor

```
{
  "pageUrl": "https://example.com/services",
  "cssSelector": "#pricing .premium-card",
  "domPath": ["main", "section:nth-of-type(3)", ".premium-card"],
  "xPercent": 63.4,
  "yPercent": 38.7,
  "viewport": { "width": 1440, "height": 900 },
  "revisionId": "rev_01j...",
  "comment": "Increase the spacing here"
}
```

Recommended Passoff sequence

Phase	Capability	Reason
1	Installed JavaScript review snippet	Fastest reliable route to genuine live-site motion, navigation and responsive behavior.
2	Screenshot fallback and visual-diff capture	Provides a safe path for sites that cannot or should not load interactively.
3	Browser extension	Supports authenticated applications and sites where the agency cannot install a snippet.
4	Public-URL proxy beta	Attempt only after customer demand justifies CSP, cookies, CORS, WebSockets, service workers and framework compatibility work.

Product decision

Do not make universal proxy compatibility a launch dependency. Let Passoff win on formal approval, revision integrity and developer handoff while the live-site capture layer matures behind proven demand.

33. Immediate next decisions

The following decisions unblock validation and implementation. Make them before broad feature design.

- 1 Confirm the launch niche: Webflow freelancers and 2-10 person web studios.
- 2 Choose the primary promise to test: approval speed, scope protection or premium handoff.
- 3 Interview 20 qualified prospects using recent-project questions.
- 4 Run five concierge projects before automating the workflow.
- 5 Choose production Postgres and object-storage providers; record the decisions in ADRs.
- 6 Define the approval statement, evidence model and legal review boundary.
- 7 Freeze the 12-week MVP scope and explicit non-goals.
- 8 Set up the activation dashboard before inviting beta customers.

Recommended first move

Create one exceptional demo room for a fictional Webflow launch, then use it during interviews. The demo should show the full story from feedback to approved revision to organized handoff - not a disconnected feature tour.

Decision summary

Decision	Recommendation
Position	Branded client approval and asset handoff room
First niche	Webflow freelancers and small web studios
Pricing	\$19 Solo, \$49 Studio, \$99 Agency; clients free
MVP	Images/PDF/screenshots, pins, revisions, approval ledger, handoff
Stack	Next.js + TypeScript + Postgres/Drizzle + private object storage
Go-to-market	Founder outreach, free toolkit, SEO and ecosystem partners
First expansion	Developer handoff generated from an approved client revision
Proof threshold	5 paying businesses, 10 real projects, 3 case studies

Sources and research notes

Published prices and feature sets change. The following first-party sources were consulted and should be rechecked before public use. Accessed August 29-30, 2026.

- [1] Frame.io pricing: <https://frame.io/pricing>
- [2] BugHerd pricing: <https://bugherd.com/pricing>
- [3] Markup.io pricing: <https://www.markup.io/pricing/>
- [4] Filestage pricing: <https://filestage.io/pricing/>
- [5] Pastel plans: <https://usepastel.com/plans>
- [6] Pastel FAQ and product capabilities: <https://usepastel.com/faq>
- [7] Ruttl pricing: <https://www.ruttl.com/pricing>
- [8] ManyRequests pricing: <https://www.manyrequests.com/pricing>
- [9] Atarim pricing: <https://atarim.io/pricing/>
- [10] Webflow Certified Partner program: <https://experts.webflow.com/apply>
- [11] Webflow Certified Partner directory: <https://webflow.com/certified-partners>
- [12] Supabase pricing: <https://supabase.com/pricing>
- [13] Vercel pricing: <https://vercel.com/pricing>
- [14] Vercel Blob documentation: <https://vercel.com/docs/vercel-blob>
- [15] Neon pricing: <https://neon.com/pricing>
- [16] Resend pricing: <https://resend.com/pricing>
- [17] Cloudflare R2 pricing: <https://developers.cloudflare.com/r2/pricing/>
- [18] Stripe pricing: <https://stripe.com/pricing>
- [19] Next.js App Router documentation: <https://nextjs.org/docs/app/getting-started>
- [20] Next.js backend-for-frontend guide: <https://nextjs.org/docs/app/guides/backend-for-frontend>
- [21] Ruttl getting started and public website review: <https://www.ruttl.com/support/get-started>
- [22] Ruttl web application review and installed script: <https://www.ruttl.com/features/review-web-applications>
- [23] Ruttl Chrome extension and compatibility notes: <https://site.dev.ruttl.com/chrome-extension/>

Research scope

This document synthesizes product strategy and implementation guidance from the supplied concept, first-party vendor pages and the observed Passoff starter stack. It is not a market-size study, legal opinion, security audit or vendor commitment.